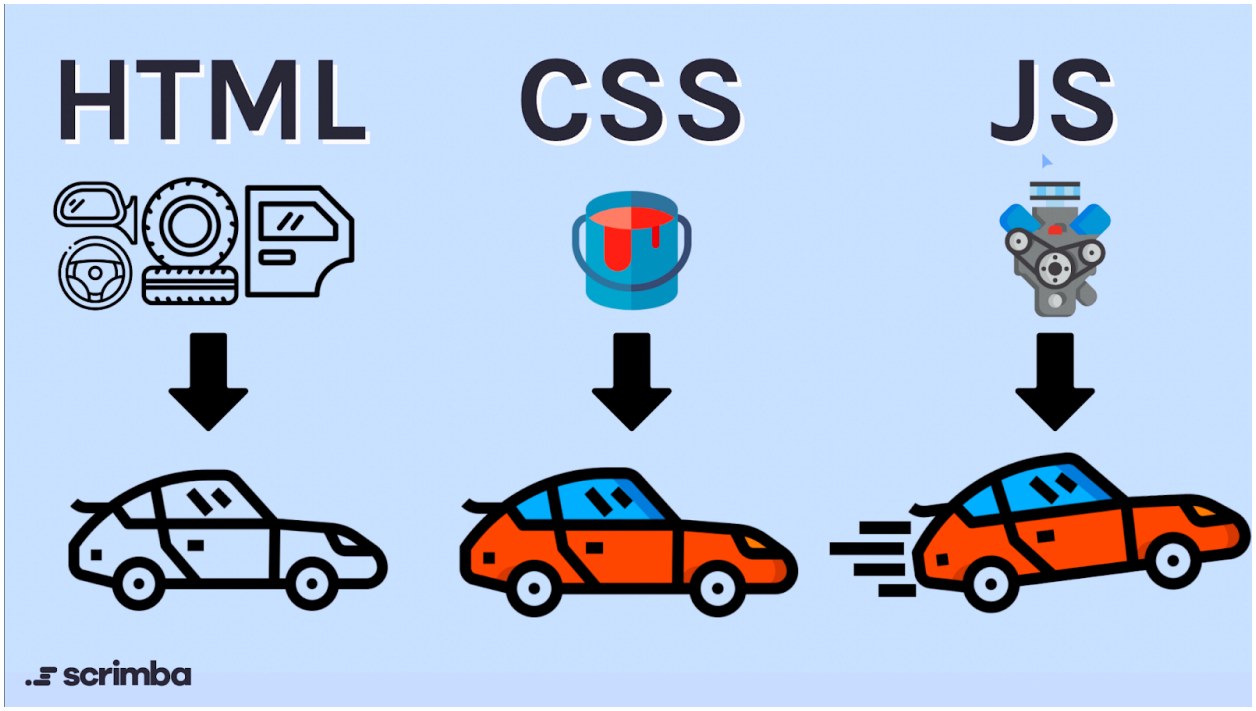
Java Script ⇒ .js ⇒ .ts

It is Programming scripting language

It give interactive ness or Functionalities of Web Pages

Java script is a high level programming scripting language and it's give more interactiveness of the web pages and web application.

We create or develop a static web application using only html,css when will you add your scripting concept of js that time only project will transforming to Dynamic web application



Importing methods 😎

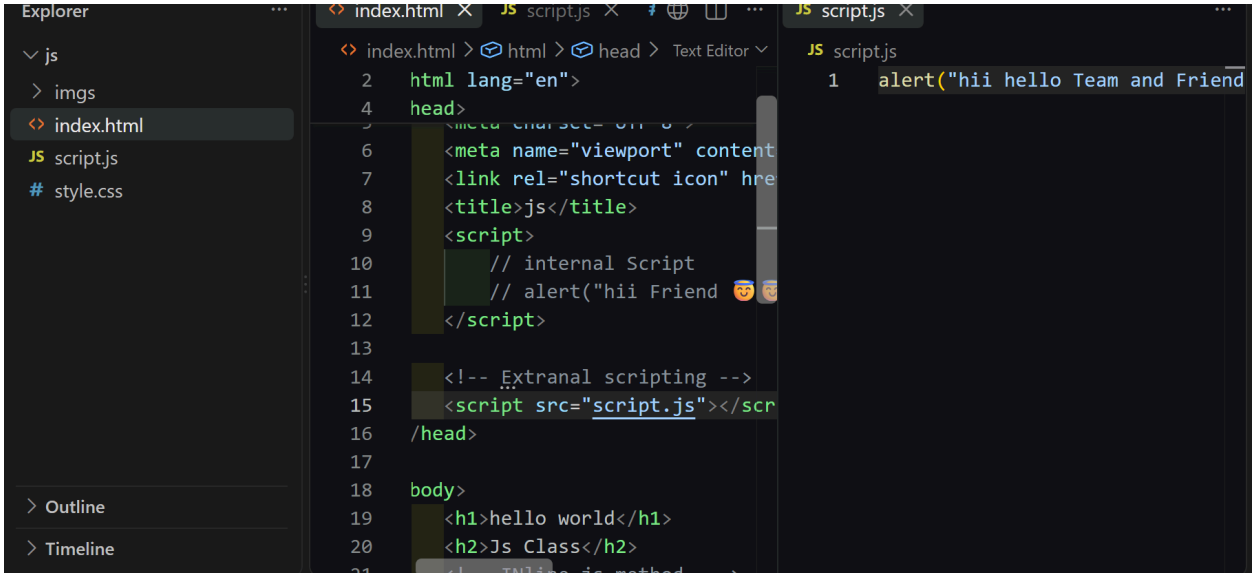
Inline Scripting method

```
<body>
    <h1>hello world</h1>
    <h2>Js Class</h2>
    <!-- INline js method  -->
    <script>
        alert("hii team")
    </script>
</body>
```

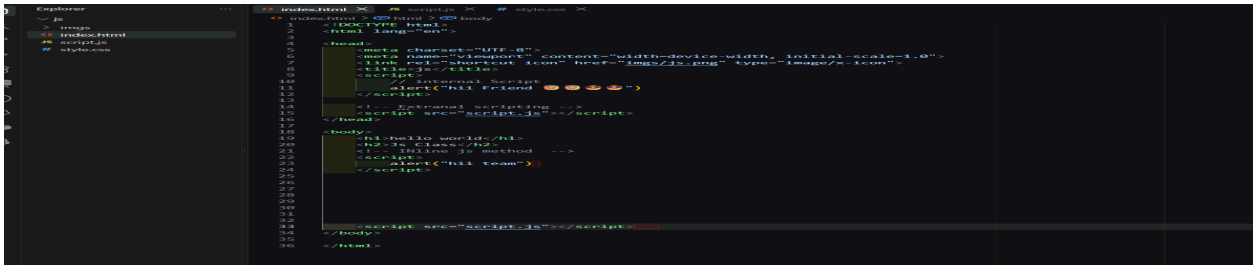
Internal Scripting method

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="imgs/js.png" type="image/x-icon">
    <title>js</title>
    <script>
        // internal Script
        alert("hii Friend 😇😇😎😎")
    </script>
</head>
```

External Scripting method



LExical Scripting method



Javascript Vs Java 👍

Java	Javascript
Core Language	Scripting language
Combination of OOPS & DSA	Optional ⇒ OOPS & DSA
Structure & Syntax or Rules we wont avoid or overcome	Structure & Syntax or Rules we will or can avoid or overcome
Backend Logics	Frontend Logic or validation
Java ⇒ oops dsa ⇒ core concepts	ECMA script ⇒ similar to java ⇒ Js Js ⇒ ECMA script 5(ES 5) , ECMA script 6 (ES 6) ⇒ es6 (js)

JS Concepts 👍

ES 5 Topics

Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
Data Type ⇒ 2 +1

Primitive Data 7	Non primitive Data 3
Number String Boolean Bigint Undefined Null Symboles	Array ⇒ =[], ⇒ = [‘ ‘,””,1,true] Object ⇒ ={}, ⇒ ={ key : value } Function

```
<script>
        //  Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Data Type  ⇒ 2 +1
        // Stractured Data type  ==> Primitive Data type (7)
        // Number  ==> [0,1,2,3,4.......n] ==> without using any
symblole  "",''
        let num1 = 100;
        const Total = 2000;
        let myNum = 10.80;
        console.log(num1);
        console.log(Total);
```

```
        console.log(myNum);
        // String ==> [a,b,c,d......z] ==> with using any symbole like "
",''
        let myName = " ganapathi";
        let myName1 = 'raja';
        console.log(myName);
        console.log(myName1);
        // Boolean ==> [true, false ==> 0,1]
        // let isValid = true;
        let isValid = false;
        console.log(isValid);
        // bigINt [ (2n*3) ==> more the 5 digits ]
        let myMobile = 75202132895
        let myValue = 100000000000000000000000
        BigInt(myMobile);
        BigInt(myValue);
        console.log(BigInt(myValue))
        // Undefined ==> [ ? ] ==> may be getting a values
        var a = 1;
        console.log(a);
        // Null ==> [ { } ]  ==> never getting a values
        var a = null;
        // symbole [ @,!,#,$,%];

        // UnStractured Data type  ==> NON - Primitive Data type (3)
    </script>
```

```
<script>

        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Array  ==> arr or [ '',"",10]
        // Hytrogenius element an dstore continuos=use loaction
        // it has two important key words
        //  index ==>start with 0==>  addresss ==> position ==> refernce
key,
        //  element ==> values ==> datas ==>
        var myArr = [100, 200, 300, "ganapathi", "sunder", "hari", true,
false, 100.200, "%^$", 7502132895, "pc", "hello", '1'];
```

```
        console.log(myArr);
        console.log(myArr[0]); // index finding
        console.log(myArr[12]);
        // console.log(myArr[13]);
        // console.log(myArr[8]);
        // console.log(myArr[8, 9, 10]);

        // Object  ==> Stand alone entity  ==> obj or  = {} or ={key :
value}

        var myCar = {
            name: "spressoo",
            color: "grey",
            price: 500000
        };

        var myCar1 = {
            name: "spressoo",
            color: "grey",
            price: 500000,
            quality: function functionality() {
                console.log(myCar1.name);
                console.log(this.color); // 3rd calling object value ==>
this lkeyword to cll inside object within the function
            }
        };
        console.log(myCar);
        console.log(myCar.name); //1st method ==>  object litral method
        console.log(myCar.color);
        console.log(`${myCar.price}`); //2nd method ==>  object
destructing method

        console.log(myCar1.quality())
        //  console.log(${ myCar.price }); // object destructing method

        // Function  ==> fnction ==> funName () { return } ==>
=()=>{return}
    </script>
```

```
<script>
        //  Array  ==> = [ ]

        let myDatas = [1, 2, 3, 4, 5, "a", "b", "c", true, false, null,
7502132896,];
        console.log(myDatas);
        console.log(myDatas[0]);
        console.log(myDatas[1]);
        console.log(myDatas[5]);

        console.log(myDatas.indexOf(true));

        // Object ==> ={ key : value  }

        const car = {
            name: "bmw",
            color: "navy",
            price: 1200000,
            model: "m5"
        }

        console.log(car)
        console.log(car.model)
        console.log(car.name) // . 1st method litral

        console.log(`${car.price}`)
        console.log(`${car.color}`) // `${}` 2nd method object
desctructing

        let waterBottle = {
            name: "jio",
            qty: "5l",
            price: 20,
            capacity: function size() {
                let smallSize = "250ml";
                let sample = this.name
                console.log(smallSize)
                console.log(waterBottle.qty)
                console.log(this.name) // this 3rd method (.this) method
```

```
                console.log(this.price) // this measn value assigner
            }
        };
        console.log(waterBottle)
        console.log(waterBottle.capacity())
        // console.log(this.AnimationTimeline) // not adviceable

        // Functions  ==> (){} or =()=>{} ==> or function
        // Reusable code (organized) collection to perform single loci
operations
        // function funName(){ return } or const funNAme =()=>{return};

        // console.log(100 + 200);



        // function ==> 2 classification
        // with Parameter function
        // parameter ==> arguments ==> reference datas  ==> props ==>
params
        function sumOftwoNumbers(a, b) { // a,b parameter ==> value
carrier
            let c = a + b;
            return c;
        }

        const sumOfNumbers = (value1, value2) => {
            return valuesAdd = value1 + value2;
        }

        console.log("nasika 100,200:  ", sumOftwoNumbers(100, 200));
        console.log("pavidra 1000,2000:  ", sumOftwoNumbers(1000,
2000));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 20));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 1));

        // without  Parameter function

        function Nasika() {
            let a = 100;
```

```
            var b = 300;
            const c = a + b;
            return c;
            console.log(c)
        };
        Nasika();
        console.log(Nasika());

        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()

        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())


        // functions 4 types
        // Function expression method
        // function FunName () { return }
        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()
        // Arrow method ==> HOF ==> Call back ==> recurssive function
==> cohansive function ==>
        // variabledeg funcName =()=>{ return }
        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())
```

```
        // IIFE
        const myMainfun = (
            function Pavidra() {
                let x = 200, y = 300;
                const z = x + y;
                console.log(z);
            }
        )();
        // Generative Method
            function* Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        }

    </script>
```

Variable ⇒ Containers (temp storage) ⇒ 3 ⇒ var ,let, const
Variable Scoping & Hoisting ⇒ { } ⇒ 3

```
<script>

        // Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Collection of Inforamtaions or detalis
        // Raw facts ==> 0,1 ==> true  false
        // Data type ==> 2
        // Stractured Data type  ==> Primitive Data type (7)
        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Variable  ⇒ Containers (temp storage ) ⇒ 3 ⇒ var ,let,
const
        // var let const
        // To store temporarly in data in Opertaion basis

        // a = "ganapathi";

        // var a = "ganapathi";
        // var a = "sundar";
```

```
        // let a = "Keerthi";
        // let a = 1000;

        // const pi = 3.14
        // const Opi = 22 / 7

        // let pi = 20;

        // const a = 100;
        // const a = "ganapathi";

        // variable  varName = values

        var obj1 = "hii"
        let obj2 = 'hello'
        const obj3 = 100;

        // Variable  ⇒ {  } == > Scoping => 3
        // global scope  ==> ES5 scope  ==> var let const
        // local  scope ==>  Block level scope ==> functional scope ==>
let const
        // lexical  scope  ==> let const


        var trainer = "ganapathi";
        function batch14() {  // Globle scope
            console.log("hii globel", trainer);
            let trainee = "pavidra";
            console.log(trainee);

            function team1() { // local scope
                console.log(trainer); // lexivcaln
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
            team1()
            console.log(trainee1)
```

```
    }

    console.log(trainer);
    batch14();
    // console.log(trainee);


    function sla() {
        var trainer = "ganapathi";
        function batch14() {
            console.log(trainer);
            let trainee = "pavidra";
            function team1() {
                console.log(trainer);
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
        }
    }

</script>
```

Var	Let	Const
Es5 version variable	Es6 version variable	Es6 version variable
Void scoping (public)	Block / functional scoping	Block / functional scoping
Global scoping variable	Local scoping variable	Local scoping variable
Read / write operation can perform	Read / write operation can perform	Read operation can perform
Changeable / Mutable	Changeable / Mutable	Non Changeable / imMutable
Scoping means ⇒ { }		
Variable Hoisting ⇒ var		

Web hoisting ⇒ Local project will migrate globalize project

```
// Variable Hoisting
        // const a = "Ganpathi";
        // let a = "Ganpathi";
        // var a = "Ganpathi";
        console.log(a);
        var a = "Ganpathi Raja";
        // let a = "Ganpathi Raja";
        // const a = "Ganpathi Raja";
```

Operators ⇒ 4 + 2
Statements ⇒ 2
Conditional statements (4)
Looping Statements (5)
INbuilt Object Function
Math object (5)
Date Object (5 to 8)
Operation
String OPerations (5 to 8)
Array Operation (28 ⇒ 15)
DOM Manipulating(5) & Selecting Elements (5)
Js I/O Methods
Input method ⇒ 2
Output Method ⇒ 4
Validation of Static Web Pages

ES 6 Topics